

DocuSnap v2 — Developer Manual

Source of truth: `CLAUDE.md` (project root). This manual expands on it with implementation detail cross-checked against the source tree as of June 2026. Where a claim could not be directly confirmed against source, it is marked **(inferred)**. Everything else is **(confirmed)** by reading the cited file.

Table of Contents

1. [Introduction](#)
 2. [Technology Stack](#)
 3. [Repository Structure](#)
 4. [Runtime Architecture](#)
 5. [End-to-End Document Lifecycle](#)
 6. [Extraction Pipeline](#)
 7. [Learning Systems](#)
 8. [Template System](#)
 9. [OCR and Preprocessing](#)
 10. [Windows / UI Surfaces](#)
 11. [Search Subsystem](#)
 12. [Database Guide](#)
 13. [Logging, Diagnostics & Debugging](#)
 14. [Testing & Regression Strategy](#)
 15. [Development Workflow Guidance](#)
 16. [Build, Packaging & Deployment](#)
 17. [Known Gotchas / Sharp Edges](#)
 18. [Where to Change Things](#)
 19. [Troubleshooting Matrix](#)
 20. [Extension Roadmap Guidance](#)
-

1. Introduction

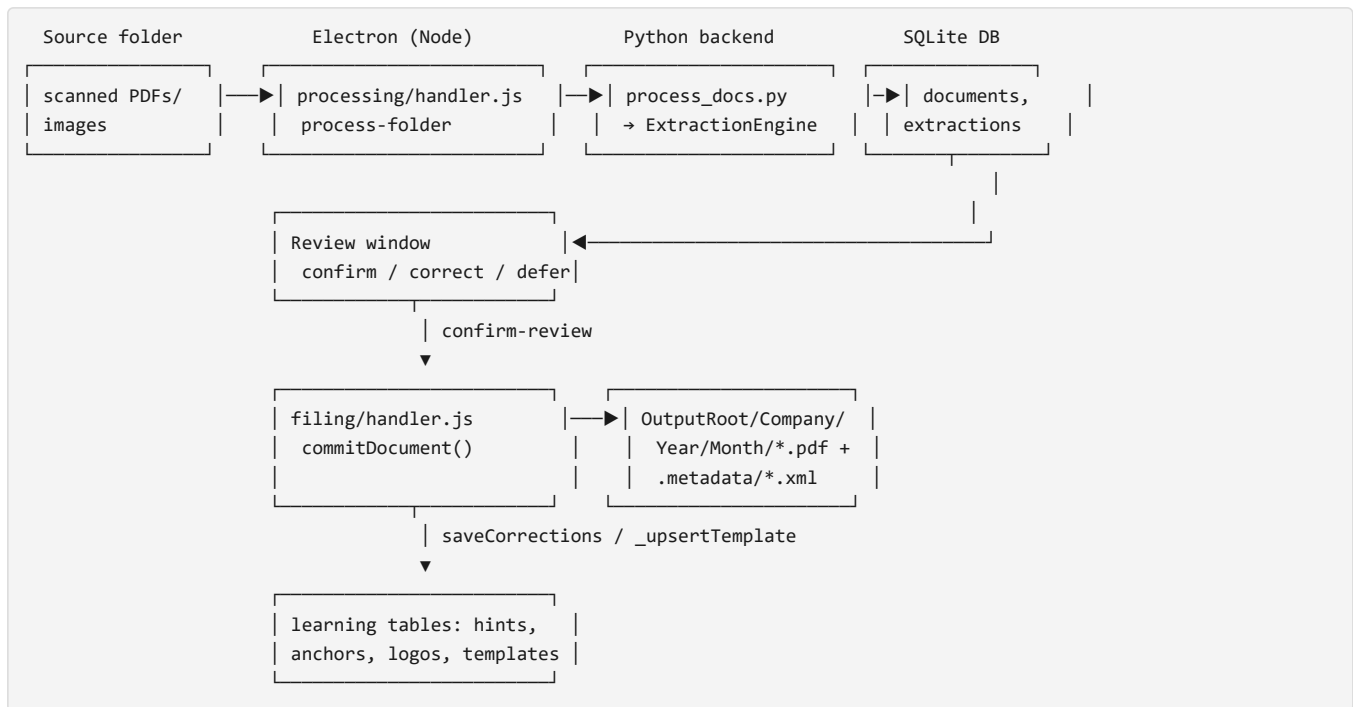
DocuSnap is a **Windows desktop application** that automates the intake of business documents (invoices, sales orders, purchase orders, and custom document types). A user points it at a folder of scanned PDFs/images; the app:

1. **OCRs** every page (Tesseract 5),
2. **Identifies the document type and supplier**,
3. **Extracts structured fields** (invoice number, date, totals, supplier name, etc.) using a multi-stage pipeline that gets smarter the more documents are confirmed,
4. Surfaces low-confidence extractions in a **Review queue** for a human to correct,
5. **Files** the confirmed document into a structured output folder (`Company/Year/Month/...`) with an XML metadata sidecar, and
6. **Learns** from every correction — anchors, supplier hints, logo fingerprints, OCR-format templates, and reusable document **templates** — so future documents from the same supplier need less (or no) manual correction.

The app is **fully offline-capable**: OCR runs locally via Tesseract, the database is a local SQLite file, and the (currently dormant) AI extraction stage uses a local Ollama model rather than a cloud API.

Platform: Windows only (desktop Electron app). Two distinct runtime contexts exist: **dev** (system Python 3.12 + system Tesseract) and **packaged** (bundled Python venv under `vendor/python/`, see [§16](#)).

High-level workflow (also see [§5](#)):



2. Technology Stack

Layer	Technology	Notes
Desktop shell	Electron 31 , Node.js	<code>package.json</code> pins electron: ^31.0.0 (confirmed)
Native DB binding	better-sqlite3 ^12.10.0	Native addon — must be rebuilt against Electron's Node ABI. This is why JS unit tests run via <code>ELECTRON_RUN_AS_NODE=1 electron <file>.js</code> rather than plain <code>node</code> (see §14)
Auth hashing	argon2 ^0.44.0	Used for password hashing in <code>database/modules/auth.js</code>
UI	Vanilla HTML/CSS/JS, frameless windows	Custom titlebar, <code>-webkit-app-region: drag</code> . No framework (no React/Vue)
OCR	Tesseract 5 via <code>pytesseract</code> + <code>pypdfium2</code>	<code>pypdfium2</code> renders PDF pages to images for OCR and for preview
AI extraction	phi3:mini via Ollama	Dormant — Stage 3 code exists (<code>llm.py</code>) but <code>ai</code> mode is not exposed in the shipped Settings UI and Ollama/the model is not bundled in the installer
Database	SQLite via better-sqlite3, WAL journal mode, foreign keys ON	Single file at <code>{userData}/docusnap.db</code>
Packaging	electron-builder ^24.13.3	NSIS installer, Windows x64 only, code signing disabled

Process roles

- **Main process** (`src/main.js` + `src/modules/*/handler.js`): owns the SQLite connection, spawns Python subprocesses, creates/manages `BrowserWindows`, enforces auth/role checks on every IPC handler.
- **Renderer processes** (`src/windows/*/renderer.js`): one per window (login, main, review, settings, search). No direct Node/DB access — everything goes through `window.docusnap` (the `contextBridge` API defined in `src/preload.js`).
- **Preload** (`src/preload.js`): the only bridge between renderer and main. `contextIsolation` is enabled; renderers cannot `require()` Node modules.
- **Python backend** (`python_backend/`): stateless CLI scripts invoked via `child_process.spawn`. Each invocation receives all the data it needs via temp JSON files (or small inline JSON args) and streams JSON lines to stdout. The Python process never touches SQLite directly — Electron reads from/writes to the DB and only passes flat data structures across the process boundary.

Dev vs. packaged differences

Aspect	Dev	Packaged
Python	py -3.12 <script>.py (system Python)	vendor/python/python.exe <script>.py (bundled venv)
Tesseract	Hardcoded C:\Program Files\Tesseract-OCR\tesseract.exe	Bundled under vendor/ (inferred — confirm path in resourcePath() / tesseractPath() in main.js before relying on it)
DB location	{userData}/docusnap.db (dev userData path)	Same mechanism, but {userData} resolves to the installed app's per-user AppData folder
__pycache__	Can go stale after edits — delete manually	Not applicable (bundled, not edited in place)

3. Repository Structure

```

docusnap2/
├─ CLAUDE.md                # Primary project memory / source of truth
├─ MODULES.md               # Older, shorter module reference (4-stage pipeline; superseded by this manual)
├─ package.json             # Electron 31, better-sqlite3, argon2, electron-builder
├─ config/
│   └─ keyword_patterns.json # document_type_keywords + field_patterns (Stage 1 regex library)
├─ templates/               # Sample/fixtures exports of DB `templates` rows (NOT loaded by the app at runtime)
├─ src/
│   ├── main.js              # Electron entry point – window creation, ctx wiring, cross-window nav
│   ├── preload.js           # contextBridge `window.docusnap` API surface
│   └─ modules/
│       ├── auth/handler.js   # Login, users, roles, audit log
│       ├── processing/handler.js # Folder import, reprocess, OCR region, logos, PDF split
│       ├── processing/processing_mode_handler.js # fast/smart mode get/set, fast-mode suggestion
│       ├── review/handler.js # Review/Deferred queues, confirm/defer/delete, page preview
│       ├── filing/handler.js  # Output folder structure, filename pattern, XML metadata
│       ├── settings/handler.js # Doc types, fields, key-value settings
│       ├── templates/handler.js # Admin Template Viewer – CRUD, mappings, groups, OCR-auto
│       ├── search/handler.js  # search-documents IPC
│       ├── watch/handler.js   # Polling folder watcher → feeds processing pipeline
│       └─ logger.js           # processing.log writer
│   └─ windows/
│       ├── login/{index.html,renderer.js} # First-run setup, login, recovery, forced reset
│       ├── main/{index.html,renderer.js}  # Folder import + run/stop + progress log
│       ├── review/{index.html,renderer.js} # Review/Deferred tabs, zone-OCR teaching tool
│       ├── settings/{index.html,renderer.js} # General/Doc Types/Fields/File Naming/Templates/Users
│       ├── search/{index.html,renderer.js,search-*.js} # Search UI (functional, not a placeholder)
│       └─ shared/theme.js           # Dark/light theme sync across windows
├─ database/
│   ├── index.js              # open(), runMigrations()/runJsMigrations(), schema helpers
│   ├── migrations/001_initial.sql # Base schema (v1)
│   └─ modules/
│       ├── document_types.js # Doc type + field CRUD, seedBuiltInTypes()
│       ├── documents.js      # Document CRUD, search(), getReviewQueue()/getDeferredQueue()
│       ├── learning.js       # Hints, anchors, logos, format collection, settings KV
│       ├── templates.js      # Template CRUD, field mappings, groups, OCR-auto params
│       ├── auth.js           # Users, recovery codes, audit log
│       └─ test_templates.js  # JS test for templates.js (run via Electron-as-Node, see §14)
├─ python_backend/
│   ├── process_docs.py        # CLI entry point – streams JSON progress to stdout
│   └─ extraction/
│       ├── engine.py          # ExtractionEngine – staged pipeline orchestration
│       ├── template_matcher.py # Stage 0: learned-template ID + field seeding
│       ├── template_mapper.py  # Stage 0.5: admin-drawn anchor→target zone mapping
│       ├── keyword.py          # Stage 1: regex pattern matching + doc-type detection
│       ├── anchor.py           # Stage 2: spatial anchors + logo supplier match
│       ├── ocr_corrector.py    # Stage 2.5: learned OCR misread correction + noise stripping
│       ├── llm.py              # Stage 3: phi3:mini via Ollama (dormant)
│       ├── validator.py        # Stage 4: cross-field validation
│       └─ format_anomaly_checker.py # Stage 4.5: format-class anomaly detection
└─ ocr/{tesseract.py,region.py} # Page OCR + zone-selection OCR

```

```

├── ocr_region.py          # CLI wrapper around ocr/region.py (used by ocr-region IPC)
├── logo/fingerprint.py   # Logo perceptual-hash extraction/matching
├── logo_fingerprint.py   # CLI wrapper around logo/fingerprint.py
├── pdf_splitter.py       # CLI for split-pdf IPC
├── render/pages.py       # PDF→PNG rendering (review/search/template preview)
├── render_pages.py       # CLI wrapper around render/pages.py
├── tests/               # Regression fixtures + unit tests (see §14)
└── dist/                # electron-builder output (not checked in)

```

4. Runtime Architecture

4.1 Process model

DocuSnap is a single-instance Electron app. The **main process** (`src/main.js`) is the only process with DB access and the only process allowed to spawn Python. Every renderer window talks to it exclusively through the `window.docusnap` bridge (`src/preload.js`), which maps ~70 named methods to `ipcRenderer.invoke/send` calls.

4.2 Shared `ctx` object

`main.js` builds one `ctx` object after `app.whenReady()` and passes it to every module's `register(ctx)` function (confirmed, `main.js` ~line 143-168). `ctx` contains:

- **IPC & DB:** `ipcMain`, `getDb` (lazy DB accessor — opens + migrates on first call)
- **Resource paths:** `resourcePath()`, `pythonExe()`, `pythonArgs()`, `tesseractPath()`, `backendScript()`, `configPath()`, `templatesDir()` — these encapsulate the dev/packaged path differences described in §2
- **Window control:** `createWindow`, `getMainWindow`, `notifyMainWindow`, `notifyAllWindows`, `windows` (a registry of open BrowserWindows by name)
- **Utilities:** `app`, `fs`, `logger`, `spawn`, `path`

Every handler module receives the *same* `ctx`, so e.g. `processing/handler.js` and `watch/handler.js` can both call the same `notifyMainWindow` and share the same `getDb()` connection (single SQLite connection, WAL mode, used from one process).

4.3 Window inventory

Window	Size	Resizable min	Created by	Access
Login	460×660	fixed (no min/maximize)	<code>app.whenReady()</code> → <code>createWindow('login', ...)</code>	unauthenticated
Main	1100×750	800×560	<code>enterMainApp()</code> after login	any signed-in user
Review	1200×800	900×600	<code>open-review-window</code> / <code>open-review-window-at</code>	admin/edit
Settings	1100×680	900×520	<code>open-settings-window</code>	admin (most panels)
Search	1200×780	1000×600	<code>open-search-window</code>	any signed-in user

All windows are **frameless**, dark background `#0c0e14`, `contextIsolation: true`, load `preload.js`, and use `icon.ico`.

4.4 Cross-window communication

Two mechanisms:

1. **Broadcast events** (`notifyMainWindow(event, payload)` / `notifyAllWindows(event, payload)`), consumed by renderers via `onXxxChanged` listeners registered in `preload.js`: - `review-count-changed(n)`, `deferred-count-changed(n)` — badge counts - `processing-mode-changed(mode)` — Settings ↔ Main mode toggle sync - `theme-changed(theme)` — dark/light sync across all open windows - `auth-session-changed(user)` — login/logout broadcast - `process-progress(msg)`, `reprocess-progress(msg)` — streamed Python progress for the Main/Review windows

2. **Pending-navigation pattern** for "Edit in Review" from Search: - `main.js` IPC `open-review-window-at(docId)`:

1. Sets module-level `pendingReviewDocId = docId`

2. Logs an audit entry `{action: 'search_open_review', target_type: 'document', target_id: docId, details: 'source:search'}`
3. Creates/focuses the Review window - Review renderer's `loadQueue()` runs as normal, then calls `getReviewTarget()` once — this IPC handler returns and **clears** `pendingReviewDocId`. If non-null, the renderer switches to that document. - This avoids a race where the Review window isn't ready yet to receive a direct "navigate to doc" push.

4.5 Auth/session model

- Single in-memory `currentSession = { id, username, displayName, role }` (or `null`) in `auth/handler.js`, shared by the whole process — **no persistence across app restarts** (confirmed). Every relaunch requires login.
- Roles: `admin`, `edit`, `readonly` (CHECK constraint on `users.role`, migration 7).
- **Enforcement boundary**: every IPC handler calls `requireLogin()` or `requireRole(...roles)` at the top of the handler body. Renderer-side hiding of buttons/panels is UI convenience only — the real gate is in the main-process handler.
- Login window ↔ Main window swap: `auth-enter-app` (send) creates the main window *then* closes the login window (and vice versa for `auth-show-login`), so the app never momentarily has zero windows (which would trigger Electron's `window-all-closed` quit logic).

4.6 Startup sequence

```
app.whenReady()
  → init logger (processing.log)
  → build ctx
  → register auth module
  → createWindow('login', LOGIN_WINDOW_OPTIONS, 'index.html')
  → register processing, review, settings, filing, search,
    processingMode, watch, templates modules
  → if watch-folder enabled in settings → resume polling
```

5. End-to-End Document Lifecycle

1. IMPORT User picks a source folder (Main window) → process-folder
|
2. OCR Python: `pypdfium2` renders pages → Tesseract OCRs each page
|
3. TYPE DETECTION `keyword.detect_document_type()` scores doc against
document_type_keywords + custom type names
|
4. EXTRACTION `ExtractionEngine.extract()` – Stages 0 → 4.5 (§6)
|
5. PERSIST `documents.insert(status='needs_review' or 'pending')`,
learning.insertExtractions() – one row per field
|
6. REVIEW Review window: `getReviewQueue()` shows `status='needs_review'`
only. User edits fields, optionally teaches anchors via
the ⊕ zone-OCR tool, picks/keeps document type.
|
7. CONFIRM confirm-review IPC:
a. `filing.commitDocument()` – copy to OutputRoot/
Company/Year/Month/DocType.DD-MM-YYYY.RefNo.pdf
+ .metadata/*.xml sidecar
b. `learning.saveCorrections()` – corrections,
supplier_hints, anchor-clearing (§7)
c. `_upsertTemplate()` – create/update learned template,
possibly reuse via logo-hash match (§8)
d. `documents.update()` – `status='confirmed'`,
stored_path, supplier_name, doc_date,
reference_number, document_type_id
e. `_scheduleSourceMove()` – deferred deletion of the
original source file (§17)
|
8. ALTERNATIVES defer-document → `status='deferred'` (Deferred tab,

```
restorable via restore-deferred)
delete-document → admin-only, deletes file + DB row
|
9. SEARCH search-documents over confirmed (+ optionally
needs_review/deferred for admin/edit) documents,
full-text via documents.ocr_text (migration 10)
```

Status values (CLAUDE.md, database/modules/documents.js): pending | needs_review | deferred | confirmed | deleted | error (plus template_sample, used only for admin-imported template sample documents — see §8).

⚠️ `getReviewQueue()` filters strictly on `status='needs_review'`. Inserting a document as `status='pending'` writes it to the DB but it will **never appear** in the Review window. If you add a new ingestion path, make sure it sets `needs_review` (or routes through the existing `_handleFileMessage` path) — see §17.

6. Extraction Pipeline

Entry point: `python_backend/process_docs.py` → `ExtractionEngine.extract()` in `python_backend/extraction/engine.py`.

6.1 Modes

Stored in `settings.processing_mode` (`fast` | `smart` | `ai`, default `smart`):

Mode	Stages run	LLM (Stage 3)
fast	0, 0.5, 1, 2, 2.5, 4, 4.5	never
smart (default)	same as fast	only if <code>invoice_number</code> / <code>invoice_date</code> / <code>total_amount</code> missing or confidence < 70%
ai	same as fast	always

`_should_use_llm()` returns `True` only in `ai` mode **and** when the LLM is available; `fast` / `smart` paths short-circuit before any Ollama call.

6.2 Stage-by-stage

Stage	Module	Purpose	Output method tags
Pre	<code>template_matcher.compute_logo_hash</code> , <code>extract_keyword_fingerprint</code>	Compute page-1 logo phash + stable keyword fingerprint (digits/calendar words/stop-words filtered out)	—
0	<code>template_matcher.identify_template</code> + <code>extract_with_template</code>	Match a learned template by logo-hash (Hamming < 13, accept ≥60 confidence) or keyword fingerprint (≥75% recall); seed fields from <code>template_fields</code>	<code>template_fixed</code> (95%), <code>template_anchor</code> (85%)
0.5	<code>template_mapper.extract_with_mappings</code>	Admin-drawn anchor→target zone mappings (Settings → Templates → "Map a Field"). Only runs if the matched template has ≥1 enabled mapping and page images are available. Relocates the anchor via fuzzy text match, derives the target from the anchor's <i>current</i> position + stored offset (handles scan drift)	<code>template_mapping</code> (≤96%), <code>template_mapping_expanded</code> (penalised –12)
Pre (fallback)	<code>anchor.try_logo_supplier_match</code>	If supplier still unknown, Hamming-match page logo against <code>logo_fingerprints</code> (threshold 12, confidence ≥60 to accept)	<code>logo</code>
1	<code>keyword.extract_fields</code>	Regex/label search using <code>config/keyword_patterns.json</code> (<code>_label_pattern</code> is whitespace-tolerant: "Purchase Order No" matches OCR'd "PURCHASE ORDERNO")	<code>keyword</code>

Stage	Module	Purpose	Output method tags
2	<code>anchor.extract_with_anchors</code>	Learned <code>field_anchors</code> : prefer image crop + re-OCR at <code>(x_norm, y_norm, w_norm, h_norm)</code> (taught via the $\oplus$ tool), else text-search by label/direction. Anchors filtered/prioritised: exact <code>(supplier, doc_type)</code> > supplier-only > doc_type-only > global	<code>anchor_crop</code> ($\leq 97\%$), <code>anchor</code> ($\leq 95\%$)
2.5a	<code>engine.py</code>	If supplier still unknown, scan first 600 chars of OCR text against confirmed <code>supplier_hints</code> (<code>usage_count</code> ≥ 3)	<code>hint_text_match</code>
2.5b	<code>engine.py</code> <code>_apply_hints</code>	Fill missing fields from confirmed, non-variable supplier hints (<code>usage_count</code> ≥ 2)	<code>hint</code>
2.5c	<code>ocr_corrector.denoise_value</code>	Strip learned noise edges (e.g. <code>"# 14269"</code> $\rightarrow$ <code>"14269"</code>) for <code>invoice_number</code> -style fields with a learned digits-only profile (≥ 10 confirmed samples)	+5 confidence boost
2.5d	<code>ocr_corrector.correct_extraction</code> / <code>try_correct</code>	Apply learned per-template character substitution maps (<code>LETTER_TO_DIGIT</code> , <code>DIGIT_TO_UPPER</code> , <code>DIGIT_TO_LOWER</code>)	appends <code>" +corrected"</code> , boost 0–20
3	<code>llm.py</code> (dormant in shipped UI)	phi3:mini fills only fields still missing	<code>llm</code>
4	<code>validator.validate_and_adjust</code>	Label-shaped-value guard (value ending <code>:</code> $\rightarrow$ cap 35), date/reference noise stripping, date parsing/normalisation to <code>DD-MM-YYYY</code> , subtotal+VAT $\approx$ total cross-check (2% tolerance, cap 50 on mismatch), date sanity (> 10 y old/future $\rightarrow$ cap 40), currency inference from symbol	various caps + <code>validation_note</code>
4.5	<code>format_anomaly_checker.check_value</code>	Compare value against the supplier/doc-type/field's inferred format class (from ≥ 3 confirmed historical samples). On violation: cap confidence at 45 , set <code>validation_note</code> , force <code>_needs_review</code> . Skips fields already flagged by Stage 4	cap 45

6.3 Merge / override algorithm (critical)

`results` is a flat dict; each stage merges into it differently:

Stage	Merge rule
0 (template)	Direct assignment, no comparison — always applies if a template matched
0.5 (mapping)	Authority override : applies if <code>existing</code> is <code>None</code> or <code>existing["method"]</code> in <code>("template_fixed", "template_anchor")</code> , or <code>data["confidence"] > existing["confidence"]</code> . A curated admin mapping always beats Stage 0's generic guess regardless of confidence numbers
1 (keyword)	Confidence comparison only: <code>data["confidence"] > existing["confidence"]</code>
2 (anchor)	Taught override : a result with <code>method == "anchor_crop"</code> overrides anything except another taught source (<code>anchor_crop</code> , <code>template_mapping</code> , <code>template_mapping_expanded</code>) — those contend on confidence. This lets a freshly-taught anchor (<code>usage_count=1</code> , $\sim 85\%$) beat an already-wrong keyword hit (88–93%)
2.5b (hints)	Fill-missing only: if not <code>existing</code> or not <code>existing.get("value")</code>
3 (LLM)	Fill-missing only
4 / 4.5 (validation)	In-place modification — may lower confidence/add <code>validation_note</code> , never raises it

6.4 Supplier identity re-resolution

`supplier_name` / `_supplier_name` is **not frozen** at Stage 0. After Stages 1 and 2 run, `engine.py` compares the provisional supplier name (from the template match or logo match) against `results['supplier_name']['value']`. If they differ, the **later** value wins — e.g. a taught `anchor_crop` reading the real page beats a near-duplicate-logo template guess. This re-resolution happens **once**, after every

stage that can touch supplier identity, and **before** any hint/anchor/logo/template persistence — otherwise the learning corpus would be silently written against a stale identity. `test_supplier_identity_stability.py` and `test_supplier_name_precedence.py` guard this behaviour.

6.5 `_`-prefixed metadata contract

`engine.extract()` returns a flat dict mixing field-result dicts with metadata keys:

```
results["_supplier_name"]      # re-resolved final identity (str | None)
results["_document_type"]     # display name
results["_document_slug"]     # e.g. "invoice" – used for all learning lookups
results["_overall_confidence"] # weighted avg over required fields
results["_needs_review"]      # bool – any field below threshold OR format anomaly
results["_mode_used"]         # "fast" | "smart" | "ai"
results["_template_id"]       # matched template id, or None
results["_logo_phash"]        # page-1 perceptual hash, or None
results["_keyword_fingerprint"] # list[str] – stable branding keywords
```

`process_docs.py` **pops every `_`-prefixed key before** calling `sanitise_extractions()`:

```
def sanitise_extractions(extractions: dict) -> dict:
    clean = {}
    for key, data in extractions.items():
        if key.startswith('_'):
            continue
        if isinstance(data, dict):
            clean[key] = data
        elif data is not None:
            clean[key] = {"value": str(data), "confidence": 50, "method": "unknown"}
        else:
            clean[key] = {"value": None, "confidence": 0, "method": "unknown"}
    return clean
```

This is the fix for **BUG 1+2** (`'str'` object has no attribute `'get'`): non-dict leftovers are normalised into proper field dicts. `validator.py` also normalises defensively as belt-and-braces. **If you add a new `_`-key, add it to the pop list in `process_docs.py` and make sure `validate_and_adjust` / `sanitise_extractions` skip it.**

6.6 Render module

`python_backend/render/pages.py` renders PDF pages to base64 PNG data URIs at 1.5× scale (108 DPI), with a `_win_long_path()` helper that prefixes `\\?\` so PDFs in folders with trailing dots/spaces (Windows silently strips these) can be opened. This single module is **shared** by: - Review window page preview (`get-document-pages`) - Search window result preview (`get-document-pages`) - Admin Template Viewer preview (mapping tool)

7. Learning Systems

DocuSnap has **four** learning corpora, all written during `confirm-review` and all read at the start of the next `process-folder` / `reprocess-document` run via `buildTrainingArgs()`:

Corpus	Table	Written by	Read by
Templates	<code>templates</code> , <code>template_fields</code> , <code>template_field_mappings</code>	<code>_upsertTemplate()</code> (review/handler.js) on every confirm; admin edits via Template Viewer	Stage 0 (<code>template_matcher</code>), Stage 0.5 (<code>template_mapper</code>)
Field anchors	<code>field_anchors</code>	<code>learning.saveAnchor()</code> — called when the user teaches a field via the ⊕ zone-OCR tool (taught fields), and indirectly cleared on manual corrections	Stage 2 (<code>anchor.extract_with_anchors</code>)
Supplier hints	<code>supplier_hints</code>	<code>learning.saveCorrections()</code> — every confirmed field value (not just	Stage 2.5a/b

Corpus	Table	Written by	Read by
		corrections) is upserted as a hint	
Logo fingerprints	logo_fingerprints	save-logo-fingerprint IPC (admin/edit, via zone-OCR/teaching flow)	Pre-stage and Stage 0 fallback supplier match
Format/OCR templates	(derived in-memory from extractions / corrections / documents, not a dedicated table)	learning.getFieldFormats() queries confirmed extractions per (supplier, doc_type, field_key)	Stage 2.5c/d (ocr_corrector), Stage 4.5 (format_anomaly_checker)

7.1 saveCorrections() — the central learning hub

database/modules/learning.js, called from confirm-review with (db, document_id, corrections, supplier_name, document_type, allValues, taughtFields).

In one transaction:

- For each entry in corrections (field_key → {original_value, corrected_value}): - Insert a row into corrections - If corrected_value is non-empty, upsert supplier_hints (both supplier-scoped and __global__-scoped) - **Anchor-clearing decision:**
 - If the field is **NOT** in taughtFields → clearAnchors(): DELETE field_anchors rows for (field_key, supplier_name OR '__unknown__' OR NULL). Rationale: a manual text correction means the *previously stored* anchor position pointed at the wrong spot — wipe it so a correct one can be relearned from scratch.
 - If the field **IS** in taughtFields → skip clearing. The ⊕ tool already called saveAnchor() for this exact field moments earlier in the same confirm cycle; clearing now would destroy a just-taught anchor before it ever survives one cycle.
- For **every** confirmed field value in allValues (corrected or not), upsert supplier_hints with usage_count++.

7.2 saveAnchor() — incremental anchor learning

database/modules/learning.js. Unique key: (supplier_name, document_type, field_key, anchor_label, direction).

- First teach:** INSERT with usage_count=1.
- Subsequent teach,** position compared against stored (x_norm, y_norm) with a tolerance derived from the stored box size (max(w_norm, h_norm)/2, floored at ANCHOR_MIN_TOLERANCE = 0.015):
- Within tolerance** → *refinement*: usage-weighted running average $new = (old * usage_count + incoming) / (usage_count + 1)$ — converges smoothly instead of jumping 50% each time.
- Outside tolerance** → *correction*: incoming position **replaces** stored position outright (the user clearly re-pinned a different spot).
- Either way: usage_count++, confidence += 0.1 (capped at 1.0), last_seen updated.

7.3 Anchor priority/filtering at extraction time

anchor.py _filter_anchors(): 1. Match rules: **exact equality** (normalised, case-insensitive), never substring — prevents a supplier hint for "P0" from matching inside "Polychemtex". 2. Priority order: exact (supplier, doc_type) > supplier-only > doc_type-only

global (supplier in {'__unknown__', '__global__', ''}). 3. Within a priority tier, sort by usage_count descending.

7.4 Logo fingerprints

learning.saveLogoFingerprint(): for a new (supplier_name, phash, ahash), scan existing fingerprints for that supplier; if Hamming distance ≤ 10, treat as the same logo (match_count++), else insert a new fingerprint row. Matching at extraction time (anchor.try_logo_supplier_match, templates.findByLogoHash) uses a looser threshold (12) and a confidence formula $\max(0, 100 - dist*6)$.

7.5 Format/OCR-correction learning (Stage 7 — partially implemented)

CLAUDE.md describes a 3-stage "field format cross-referencing" feature:

- Stage 1 (COMPLETE):** format_anomaly_checker.py (Stage 4.5 in engine.py), getFieldFormats() (recency-ordered sample collection), migration 11 added extractions.validation_note. Format classes: digits_only | upper_alphanum | alphanum |

alphanum_sep | date_like | currency_like | freetext. Requires ≥ 3 distinct confirmed values per (supplier_name, document_type, field_key); disagreement among samples $\rightarrow$ freetext (no constraint applied).

- **Stage 2 (COMPLETE):** propose_correction() proposes corrections via the same LETTER_TO_DIGIT / DIGIT_TO_UPPER maps used by ocr_corrector, only for digits_only fields, only when ≤ 2 chars changed and $\leq 25\%$ of the value length affected. **Never silently applied** — display_value unchanged, corrected_to holds the candidate, was_corrected stays False, _needs_review forced, validation_note = "format anomaly: correction candidate – {corrected_to}".
- **Stage 3 (NOT IMPLEMENTED):** CLAUDE.md describes a planned field_format_rules table (migration 12) for a *persistent* learned format model read via --format-rules-file. **Migration 12 was actually used for template-level OCR auto-processing** (templates.ocr_auto_enabled / ocr_auto_params — see §8), not for field_format_rules. As of this snapshot, field_format_rules does not exist in any migration, and getFieldFormats() only collects samples in-memory per run — it does not persist a learned model. **Treat Stage 3 as fully unimplemented**, not "started under a different migration number".

7.6 Audit log

database/modules/auth.js audit_log table (migration 7): (user_id, action, target_type, target_id, details, created_at). auth/handler.js exports logAudit(db, entry) which auto-injects currentSession?.id. Used for auth events (login_success, login_failure, password_reset, recovery_code_issued, recovery_code_use, role_change, user_enabled / user_disabled, user_created) and for search_open_review (logged from main.js when "Edit in Review" is used from the Search window).

8. Template System

A **template** is a reusable description of one supplier+document-type layout: a logo fingerprint, a stable keyword fingerprint, a set of fixed/anchor field rules (template_fields), and optionally admin-drawn zone mappings (template_field_mappings).

8.1 Creation & update — _upsertTemplate()

src/modules/review/handler.js, called from confirm-review:

1. Build field rules via _buildTemplateFields(allValues, dtInfo): - **Variable fields** (reference numbers, dates — field.is_variable true or unspecified): {field_key, is_variable: true, fixed_value: null, anchor_label: null}. These are taught per-document via the $\oplus$ zone-OCR tool (Stage 2 anchors), not via the template. - **Constant fields** (company name/address etc.): {field_key, is_variable: false, fixed_value: <confirmed value>, anchor_label: null}.
2. **Reuse vs. create:** - If the document already has a template_id, call templates.update(db, templateId, {logo_phash, keyword_fingerprint, fields}) — confirmed_count++. - Else, if the document has a logo_phash, try templates.findByLogoHash(db, logo_phash). **Reuse only if confidence ≥ 60** — the same acceptance gate Stage 0 (template_matcher.identify_template) uses for logo matches, so a document is never *learned into* a template that the matcher itself wouldn't have matched it to. - Otherwise create a brand-new template (name derived from supplier + doc type, templates.create(...)), and set the document's template_id.

8.2 Sample-document linkage

templates.sample_document_id (migration 8) points at one confirmed document used for **preview** in the Admin Template Viewer. get-template-sample-candidates(templateId) returns up to 30 confirmed documents with template_id = templateId, ordered by confirmed_at DESC. For brand-new templates with no confirmed documents yet (chicken-and-egg), admins can pick-template-sample-file + import-template-sample-file, which inserts a document with status = 'template_sample' — a status deliberately excluded from getReviewQueue, getDeferredQueue, all counts, and search(), so it never leaks into normal workflows.

8.3 Matching, grouping & the OCR-auto guardrail

- **Matching** (template_matcher.identify_template): logo-hash first (Hamming < 13 , accept if confidence ≥ 60), else keyword-fingerprint ($\geq 75\%$ recall against the stored fingerprint).
- **Listing/grouping:** templates.getAll() orders by confirmed_count DESC, name. template_groups (migration 9) is **organisational metadata only** (v1) — group_id does not affect matching order or eligibility.
- **Audited guardrail (Phase F-G of this project):** ocr_auto_enabled / ocr_auto_params (migration 12, see §9) are **not read** by template_matcher.py or by Stage 0 in engine.py, and templates.getAll()'s ordering is independent of these columns. **OCR auto-processing state must never influence template match priority or candidate ordering.** This was explicitly audited

and is enforced by `python_backend/tests/test_template_matcher.py`. If you touch matching logic or the templates query, re-check this invariant.

- In the Settings → Templates grouped list, each template shows a small ⚡ **indicator** when `ocr_auto_enabled` is true (Phase F/G UI addition, `src/windows/settings/index.html` + `renderer.js`) — display-only, does not feed back into matching.

8.4 Stage 0.5 — admin-drawn anchor→target mappings

`template_field_mappings` (migration 8), edited via Settings → Templates → "Map a Field":

Column	Meaning
<code>anchor_text</code> , <code>anchor_x/y/w/h_norm</code>	Where the anchor label sits on the sample page (normalised 0–1 coords)
<code>target_x/y/w/h_norm</code>	Where the value sits on the sample page
<code>offset_dx/dy_norm</code>	<code>target</code> - <code>anchor</code> — computed once at save time (<code>templates.saveMapping</code>)
<code>ocr_type</code>	hint for crop/clean behaviour (<code>text</code> , <code>multiline_text</code> , etc.)
<code>search_expansion</code>	how much to widen the anchor search box if the exact text isn't found
<code>region_hint</code>	precomputed 8-cell grid (2×4) the target overlaps — an optimisation hint
<code>enabled</code>	per-mapping toggle
<code>last_test_*</code>	result of the most recent "test mapping" run, persisted across reloads

At extraction time (`template_mapper.extract_with_mappings`), the mapping is **not** replayed at its saved absolute coordinates. Instead:
1. Locate the anchor text near its saved position (with `search_expansion` widening if needed) via fuzzy line matching. 2. Derive the target as `located_anchor_position` + (`offset_dx_norm`, `offset_dy_norm`). 3. Crop and OCR the derived target.

This makes mappings tolerant of scan/print drift between the sample document and future documents of the same supplier. See §6.2 and §6.3 for how mapping results are merged (curated-refinement override of Stage 0's generic guesses).

8.5 Preview rendering

The Admin Template Viewer's page preview (for drawing anchor/target boxes) uses the same `render/pages.py` module as the Review and Search windows (`get-document-pages` IPC) — see §6.6.

9. OCR and Preprocessing

9.1 Engine path

- **Page rendering:** `pypdfium2` rasterises PDF pages to images.
- **OCR:** `pytesseract` wraps Tesseract 5. In dev, the binary path is hardcoded to `C:\Program Files\Tesseract-OCR\tesseract.exe` (`tesseractPath()` in `main.js`'s `ctx`); packaged builds use a bundled path under `vendor/`.
- **Zone OCR:** `python_backend/ocr/region.py` (CLI wrapper `ocr_region.py`) — OCRs a single cropped region, used by:
 - The ⊕ teaching tool in the Review window (`ocr-region` IPC)
 - `template_mapper.py`'s anchor relocation and target crop+OCR

9.2 Review-window OCR enhancement workflow

`get-enhanced-preview({folderPath, filename, page, enhanceParams})` (`review/handler.js`): writes `enhanceParams` (skew/threshold/noise sliders) to a temp JSON file, spawns `render/preview_enhance.py` with `--file --page --enhance-file`, returns the enhanced page image. This is the "OCR Preview" toggle in the Review window — purely a **preview**, it does not by itself change what gets stored.

9.3 Reprocess behaviour & ephemeral vs. persistent params

`reprocess-document({docId, folderPath, filename, enhanceParams?})` (`processing/handler.js`):

- If `enhanceParams` is supplied (OCR Preview was active when the user hit Reprocess), it's used as a **one-shot override** for this run.

- If the document also has a `template_id`, the *same* `enhanceParams` are persisted as the template's baseline via `templates.setOcrAutoParams(db, templateId, enhanceParams)` — this sets `ocr_auto_enabled=1` and stores `ocr_auto_params` (JSON `{skew, threshold, noise}`).
- If `enhanceParams` is **not** supplied but the document's template has `ocr_auto_enabled` && `ocr_auto_params`, those stored params are used as the baseline automatically — no manual OCR Preview needed.
- Result merge after reprocess: for each field, **keep the existing extraction if its confidence is higher** than the new run's; if the new run found nothing for a field the old run had, **preserve the old value** (prevents silent field loss on a noisy reprocess).

9.4 Template-level auto-processing toggle

`set-template-ocr-auto(templateId, enabled)` (admin) flips `ocr_auto_enabled` without discarding `ocr_auto_params` — disabling and re-enabling restores the previously learned baseline.

9.5 Overprocessing risks

- Aggressive denoise/threshold params tuned for one noisy scan can *hurt* cleaner scans from the same supplier if persisted as the template baseline — there is currently no automatic A/B or rollback; an admin must manually toggle `ocr_auto_enabled` off or re-teach via OCR Preview + Reprocess.
- `ocr_auto_params` is supplier/template-scoped, never global — a bad rule for one supplier cannot affect another's extraction (consistent with the strict-scoping principle used throughout the learning systems).
- Reminder from 8.3: whatever preprocessing rules exist, they must stay **decoupled from template matching/ordering**.

10. Windows / UI Surfaces

All windows share: frameless chrome (`#0c0e14` background, custom titlebar/drag region), `src/windows/shared/theme.js` (dark/light, synced via `settings.theme` + `theme-changed` broadcast), and the `window.docuSnap` bridge from `preload.js`.

10.1 Login (`src/windows/login/`)

Seven screens toggled via `showScreen(id)`:

Screen	Purpose	Key API calls
<code>screen-loading</code>	Initial brand splash while <code>auth-get-status</code> resolves	<code>authGetStatus</code>
<code>screen-setup</code>	First-run only — create initial admin	<code>authFirstRunSetup</code>
<code>screen-recovery-code</code>	Show one-time recovery code (post-setup or post-recovery); requires "I saved it" ack before continuing — no way back	—
<code>screen-login</code>	Normal username/password login	<code>authLogin</code>
<code>screen-must-change</code>	Forced password reset (<code>must_change_password=1</code>) — no "current password" field	<code>authSetNewPasswordAfterReset</code>
<code>screen-forgot</code>	Explains admin-reset vs. solo-admin recovery-code paths	—
<code>screen-recover-admin</code>	Recovery code + new password	<code>authRecoverAdmin</code>

Login is rate-limited per-username with a progressive delay ladder `[0, 0, 1s, 2s, 5s, 15s, 30s]`, and uses a dummy-hash check for unknown usernames so failed-login timing doesn't reveal whether an account exists.

10.2 Main window (`src/windows/main/`)

Folder-import UI: folder picker → Run/Stop/Clear → progress log + results table + stage indicator (`ocr / llm / save / done`). Listens to `process-progress` events (`start`, `file_begin`, `file_done`, `log`) via `onProgress`. Run button disabled while `running`.

10.3 Review window (`src/windows/review/`)

Two tabs — **Review Queue** and **Deferred** — with badges fed by `review-count-changed` / `deferred-count-changed`.

- `loadQueue()`: fetches `getReviewQueue()`, `getDeferredQueue()`, `getAllDocTypes()`; populates the document-type dropdown; renders both lists; selects the first doc; then calls `getReviewTarget()` once to honor a pending "Edit in Review" navigation from Search (see §4.4).
- Document-type dropdown: changing it reloads the field list for that type while preserving already-typed values (`FALLBACK_FIELD_KEYS = ['supplier_name', 'invoice_number', 'invoice_date']` used before a type is picked).
- Per-field zone-OCR teaching ($\oplus$ tool): user drags a box on the page image $\rightarrow$ `ocr-region` $\rightarrow$ field value filled, `anchorTaughtFields` tracked $\rightarrow$ on confirm, `save-field-anchor` was already called and the field key is passed in `taught_fields` so `saveCorrections()` doesn't immediately clear the anchor it just learned (see §7.1).
- OCR Preview toggle drives `get-enhanced-preview` for live preview; Reprocess passes the active `enhanceParams` through (see §9.3).
- Deferred tab: each row has **Review** ($\rightarrow$ `restoreDeferred`, switch to Review tab, load doc) and **Delete**.

10.4 Settings window (`src/windows/settings/`)

Tabs: **General** | **Document Types** | **Fields** | **File Naming** | **Templates** | **Users**.

- **General**: output folder (`pick-output-folder`), processed folder, watch folder (toggle + picker — see §17 for watch-folder caveats), theme toggle, global confidence threshold, processing mode (fast/smart — AI mode UI removed from shipped build per CLAUDE.md).
- **Document Types**: list from `get-all-doc-types-all`, enable/disable, add custom type, click $\rightarrow$ Fields sub-panel.
- **Fields**: per-type field list, add/edit (label, key auto-from-label, type, required, confidence threshold), delete (custom fields only — built-in fields show a lock icon), reorder.
- **File Naming**: pattern input (default `{docType}.{date}.{ref}`), token reference list, live preview via `preview-filename-pattern` (uses a sample value with special characters to demonstrate illegal-character stripping), "Reset to Default".
- **Templates** (Admin Template Viewer): browse/group templates, pick/import sample documents, draw anchor $\rightarrow$ target mappings, enable/disable/test mappings, toggle OCR-auto, manage template groups. Per-template ⚡ indicator shows `ocr_auto_enabled` (Phase F/G).
- **Users**: user list, create/disable/role-change/reset-password, audit log viewer (admin-only).

10.5 Search window (`src/windows/search/`)

Fully functional (not a placeholder) — `index.html` + `search-state.js` / `search-query.js` / `search-results.js` / `search-preview.js` / `search-actions.js`.

- Search bar: company, reference, full-text, date range, doc-type dropdown, "Include unconfirmed" checkbox (admin/edit only).
- Left pane: result list grouped **Confirmed** / **Uncommitted** (needs_review + deferred).
- Right pane: multi-page preview (page nav ◀▶) + field values + actions:
- Confirmed: "Open in Explorer", "Open File"
- Uncommitted: **"Edit in Review"** $\rightarrow$ `openReviewWindowAt(doc.id)` (logs `search_open_review` audit entry, see §4.4)
- A "Workflow" section exists as a **placeholder** for future approval/status actions (see §20).

11. Search Subsystem

11.1 Scope

`src/modules/search/handler.js` registers a single handler, `search-documents(params)`, available to **any signed-in user**. It always searches `status='confirmed'` documents; it additionally searches `needs_review` / `deferred` documents **only if** `params.includeUncommitted` is true **and** the caller's role is `admin` / `edit`.

11.2 Files / modules involved

File	Role
<code>src/modules/search/handler.js</code>	IPC handler, role gating, calls <code>documents.search()</code>
<code>database/modules/documents.js</code> <code>search()</code>	Builds the SQL query (filters: <code>company</code> LIKE <code>supplier_name</code> , <code>reference</code> LIKE <code>reference_number</code> , <code>dateFrom</code> / <code>dateTo</code> on <code>doc_date</code> , <code>docType</code> joins <code>document_types.slug</code> , <code>fullText</code> LIKE <code>documents.ocr_text</code>); ORDER BY <code>confirmed_at</code> DESC, <code>processed_at</code> DESC LIMIT 200

File	Role
src/windows/search/*	UI — see §10.5
src/modules/review/handler.js get-document-with-extractions, get-document-pages	Shared with Review window — Search's preview pane reuses these handlers verbatim
src/main.js open-review-window-at / get-review-target	"Edit in Review" cross-window navigation

11.3 Interaction with Review/Documents

Search never writes to `documents / extractions` directly — all mutation flows through the Review window via "Edit in Review". This keeps the confirm/correction/learning logic (corrections, anchors, hints, templates) in one place (§7).

11.4 Planned: detached search client

CLAUDE.md's roadmap mentions a future **detached search client** — a separate process/UI (possibly a lightweight web client) that queries documents **read-only**, for users who only need to find/retrieve filed documents without running the full desktop app. Today, `search-documents` is already read-mostly and role-gated, which is a reasonable seam:

- **Safe extension points:** `search-documents`'s param shape and return shape (`{confirmed, uncommitted}`) could become the contract for a small HTTP/IPC-over-socket API without changing `documents.search()` itself. `get-document-pages / get-document-with-extractions` would need the same treatment for preview support.
- **What would need care:** today these handlers assume an Electron `ipcMain` context and an in-process `currentSession`. A detached client would need its own auth/session transport — do not casually relax the `requireRole` checks to "fix" this; design a parallel auth boundary instead.

12. Database Guide

12.1 Connection & migrations

`database/index.js` : - `open()` : opens (memoised singleton) `{userData}/docusnap.db`, sets `PRAGMA journal_mode = WAL` and `PRAGMA foreign_keys = ON`, then runs `runMigrations()` and `seedDefaults()`. - `runMigrations()` : applies `database/migrations/001_initial.sql` (base v1 schema) plus tracks applied versions in the `migrations` table (`id`, `version`, `applied_at`). - `runJsMigrations(db, applied)` : runs **12 numbered JS migrations** in order, each gated on `applied.has(n)` :

#	Adds / changes
2	Upgrades v1 <code>fields</code> table to v2 (adds <code>document_type_id</code> FK); seeds <code>document_types</code> ; old table kept as <code>fields_v1_backup</code>
3	Trims built-in fields to just name/date/ref per doc type (<code>invoice</code> , <code>sales_order</code> , <code>purchase_order</code>)
4	Creates <code>templates</code> + <code>template_fields</code> tables; adds <code>template_id</code> , <code>logo_phash</code> , <code>keyword_fingerprint</code> to <code>documents</code>
5	Deletes pre-existing <code>field_anchors</code> rows where <code>field_key='customer_name'</code> (anchors that predate supplier identification and can never match)
6	Adds <code>w_norm / h_norm</code> to <code>field_anchors</code> (exact taught box size, not a fixed default)
7	Local auth: <code>users</code> , <code>recovery_codes</code> , <code>audit_log</code> tables
8	Admin Template Viewer: <code>templates.sample_document_id</code> ; new <code>template_field_mappings</code> table
9	<code>template_groups</code> table; <code>templates.group_id</code>
10	<code>documents.ocr_text</code> (up to 50,000 chars) for full-text search
11	<code>extractions.validation_note</code> (Stage 4.5 anomaly reason)
12	<code>templates.ocr_auto_enabled / templates.ocr_auto_params</code> (template-level OCR preprocessing baseline)

Helper functions: `hasColumn(db, table, column)`, `tableExists(db, table)`, `upgradeFieldsTable(db)`, `addMissingColumns(db)` (creates `settings`, `document_types`, `field_anchors`, `logo_fingerprints` if absent).

⚠ `field_format_rules` (CLAUDE.md Stage 7 / Stage 3) does not exist. Migration 12 was used for OCR auto-processing instead. If you implement Stage 3, it will need a **new migration (13)** — do not assume migration 12 covers it.

12.2 Table reference

Table	Key columns	Written by	Read by
document_types	name, slug, built_in, enabled, ref_field_key, date_field_key, sort_order	settings/handler.js (admin CRUD), seedBuiltInTypes()	Settings UI, Review type dropdown, process_docs.py (doc-type detection seed list)
fields	document_type_id (FK), key, label, type, required, built_in, enabled, confidence_threshold, sort_order	settings/handler.js	extraction field_defs, Review/Settings field lists
documents	original_filename, folder_path, document_type_id (FK), supplier_name, overall_confidence, status, template_id (FK), logo_phash, keyword_fingerprint, ocr_text, stored_filename, stored_path, doc_date, reference_number, confirmed_at, error_message	processing/handler.js (_handleFileMessage insert), review/handler.js (confirm-review, defer, delete)	Review/Deferred queues, Search, Template sample candidates
extractions	document_id (FK), field_key, raw_value, display_value, confidence, extraction_method, validation_note, was_corrected, corrected_to	learning.insertExtractions() (insert), reprocess-document (delete+reinsert with merge)	Review window field rendering
corrections	document_id (FK), field_key, original_value, corrected_value, supplier_name, document_type	learning.saveCorrections()	(learning audit trail; not read by extraction directly)
supplier_hints	supplier_name, document_type, field_key, hint_value, usage_count, last_seen (UNIQUE on first 4)	learning.saveCorrections()	Stage 2.5a/b
field_anchors	supplier_name, document_type, field_key, anchor_label, direction, page_zone, x_norm, y_norm, w_norm, h_norm, usage_count, confidence, last_seen (UNIQUE on supplier/doctype/field/label/direction)	learning.saveAnchor()	Stage 2
logo_fingerprints	supplier_name, phash, ahash, crop_zone, match_count, last_seen	save-logo-fingerprint IPC	Pre-stage/Stage 0 logo match, templates.findByLogoHash
settings	key (PK), value, updated_at	learning.setSetting()	get-setting, theme sync, processing mode, output/processed/watch folders, filename pattern
templates	name, slug (UNIQUE), document_type_slug, logo_phash, keyword_fingerprint, confirmed_count, sample_document_id (FK), group_id (FK), ocr_auto_enabled, ocr_auto_params	_upsertTemplate(), Template Viewer handlers	Stage 0, Stage 0.5, reprocess OCR-auto resolution
template_fields	template_id (FK), field_key, anchor_label, direction,	templates._upsertFields()	Stage 0 extract_with_template

Table	Key columns	Written by	Read by
	fixed_value, is_variable (UNIQUE on template_id+field_key)		
template_field_mappings	see §8.4	templates.saveMapping()	Stage 0.5
template_groups	name (UNIQUE)	Template Viewer	Template Viewer grouping UI only
users	username (UNIQUE COLLATE NOCASE), display_name, password_hash, role CHECK(admin/edit/readonly), is_active, must_change_password, last_login_at	auth/handler.js	every IPC handler's requireRole
recovery_codes	user_id (FK), code_hash, is_used, used_at	auth/handler.js	admin recovery flow
audit_log	user_id (FK, ON DELETE SET NULL), action, target_type, target_id, details	logAudit()	Settings → Users → audit log viewer
migrations	version (UNIQUE), applied_at	runMigrations / runJsMigrations	startup migration gating

12.3 Settings KV usage

Known settings keys (non-exhaustive, confirmed via handler grep): output_folder, processed_folder, watch_folder + watch_folder_enabled, processing_mode, theme, filename_pattern, plus any global-confidence threshold setting exposed in the General tab.

13. Logging, Diagnostics & Debugging

13.1 processing.log

src/modules/logger.js writes to processing.log — location is {userData}/processing.log when packaged, project root when running from source (per CLAUDE.md/main.js init). This captures: - Python stdout log messages ({ "type": "log", "text": "...", "level": "warn"|"err"|""}) - _handleFileMessage per-field extraction summaries - Reprocess merge decisions (per-field old vs. new confidence) - Watch-folder poll/queue decisions

13.2 What does *not* go to processing.log

- **Renderer-side errors** (JS exceptions in */renderer.js, IPC promise rejections surfaced as UI errors) — these only appear in DevTools console. Always open DevTools (Ctrl+Shift+I in dev, or check if a hidden webContents.openDevTools() call exists) when a UI action silently does nothing.
- **Main-process exceptions thrown before the logger is initialised** (very early startup) — check the terminal stdout/stderr where npm start was launched.
- **better-sqlite3 errors** — these throw synchronously in the IPC handler; they propagate to the renderer as a rejected promise but are **not** separately logged unless the handler wraps them in a try/catch that calls logger.

13.3 Temp-file patterns

Per CLAUDE.md, all large payloads to Python go through os.tmpdir()/ds_<name>_<timestamp>.json, cleaned up in the process's on('close') handler. If a Python invocation crashes before producing output, **stale ds_*.json files can accumulate in %TEMP%** — useful for manually inspecting exactly what was sent to a given Python script (--fields-file, --hints-file, --anchors-file, --logos-file, --templates-file, --formats-file, --enhance-file, etc.).

13.4 Stale __pycache__

If a Python source change doesn't seem to take effect, delete python_backend/**/__pycache__. (Note: the repo currently has a tracked python_backend/__pycache__/process_docs.cpython-312.pyc showing as modified in git status — this is almost certainly accidental; consider .gitignore-ing __pycache__ rather than committing compiled bytecode.)

13.5 DB inspection workflow

- DB file: `{userData}/docusnap.db` (dev: your Windows user's AppData; exact path comes from Electron's `app.getPath('userData')`).
- To **reset** during development: close the app, delete `docusnap.db` (and `-wal` / `-shm` sidecar files if present, since WAL mode is on).
- For ad-hoc inspection, use any SQLite browser, or run a short Node script with `ELECTRON_RUN_AS_NODE=1 electron -e "..."` so `better-sqlite3`'s native binding (built against Electron's ABI) loads correctly — plain `node` will fail with an ABI mismatch.

13.6 Common failure classes

Symptom	Likely cause	Where to look
'str' object has no attribute 'get'	A <code>_</code> -prefixed metadata key wasn't popped before <code>sanitise_extractions() / validate_and_adjust()</code>	<code>process_docs.py</code> pop list, <code>validator.py</code> normalisation (BUG 1+2, \$6.5)
Confirmed doc never appears in Review	Inserted with <code>status='pending'</code> instead of <code>'needs_review'</code>	<code>_handleFileMessage</code> insert call (\$5)
Anchor "doesn't stick" after teaching	<code>taughtFields</code> not passed/matched in <code>confirm-review</code> payload, so <code>saveCorrections()</code> clears the anchor it just learned	<code>review/handler.js</code> confirm payload, <code>learning.js</code> <code>saveCorrections</code> (\$7.1)
Locked-file error deleting source after confirm	Preview pipeline still has the file open	<code>_pendingSourceMove</code> / <code>_scheduleSourceMove</code> / <code>_runPendingSourceMove</code> (\$17)
PDF preview fails for a supplier folder ending in <code>./space</code>	Win32 trailing-dot/space path stripping	<code>render/pages.py</code> <code>_win_long_path()</code> , <code>test_pages_long_path.py</code>
Date regex throws "bad character range"	<code>config/keyword_patterns.json</code> <code>validation_patterns.date</code> has <code>[/-\.]</code> instead of <code>[/\-.]</code> (BUG 3)	<code>config/keyword_patterns.json</code>

14. Testing & Regression Strategy

14.1 Python regression tests (`python_backend/tests/`)

File	Tests
<code>test_format_anomaly_checker.py</code>	Stage 4.5 format-class inference (<code>build_format_class_index</code> , <code>classify_format</code>) and anomaly detection (<code>check_value</code>) across the 7 format classes
<code>test_ocr_no_plaintext.py</code>	<code>process_docs.py</code> never writes <code>{stem}_ocr.txt</code> plaintext OCR dumps into source folders (a previously-fixed leak)
<code>test_pages_long_path.py</code>	<code>render/pages.py</code> 's <code>_win_long_path()</code> Win32 trailing-dot/space fix
<code>test_pdf_splitter.py</code>	<code>pdf_splitter.py</code> range parsing and split correctness/CLI errors

File	Tests
test_supplier_identity_stability.py	Re-running extraction twice on the same document yields the same <code>_supplier_name</code> , matching the anchor-derived value rather than an earlier Stage 0 template guess (§6.4)
test_supplier_name_precedence.py	Stage 0.5 admin mappings outrank <code>template_fixed / template_anchor</code> (confidence 90 beats fixed 95) and survive <code>anchor_crop</code> override per the curated-refinement rule (§6.3)
test_template_mapper.py	Stage 0.5 geometry/offset math, fuzzy anchor relocation, low-confidence expansion retry (mocked OCR)
test_template_matcher.py	Stage 0 logo-hash (Hamming $\leq 5 \rightarrow$ confidence ≥ 65 wins immediately) and keyword-fingerprint ($\geq 75\%$ recall) matching; variable tokens excluded from learned fingerprints; OCR-auto fields don't affect matching (§8.3)
test_validator_label_guard.py	Stage 4 down-weights values ending in <code>:</code> (anchor landed on a label, not a value)
test_validator_ocr_sanitisation.py	Stage 4 date/reference noise stripping

Runner: `python_backend/tests/run_regression.py` feeds `fixtures/*.json` (synthetic OCR text + field defs/hints/anchors/templates + expected values) directly into `ExtractionEngine.extract()` with `page_images=[]` — bypassing Tesseract/rendering entirely, so tests run fast and deterministically. Exit 0 = all pass; non-zero + diff printout on regression. Run with: `py -3.12 python_backend/tests/run_regression.py`.

Per `python_backend/tests/README.md`: fixtures should document **broad failure classes**, not one-off documents — when adding a fixture for a bug, generalise the OCR text/expectation to the *pattern* that failed, not just the literal sample.

14.2 JavaScript tests

Not wired into `npm test` (no such script in `package.json`). Run directly via Electron-as-Node because `better-sqlite3` is a native addon built against Electron's Node ABI:

```
ELECTRON_RUN_AS_NODE=1 node_modules/.bin/electron src/modules/auth/test_auth.js
ELECTRON_RUN_AS_NODE=1 node_modules/.bin/electron database/modules/test_templates.js
```

- `src/modules/auth/test_auth.js`: exercises the real `auth/handler.js` + `database/modules/auth.js` against an in-memory DB with the migration-7 schema (argon2 hashing included).
- `database/modules/test_templates.js`: exercises `templates.js` `create/rename/remove` against an in-memory DB with migrations 4 & 8 (including `ON DELETE CASCADE`), verifying slug derivation, metadata-only rename, and scoped delete with no spillover into other tables.
- Other JS tests exist under `src/modules/filing/` (`test_filename_pattern.js`, `test_remove_source_file.js`) following the same pattern.

14.3 What to test for each kind of change

Change type	Minimum test coverage
New/changed extraction field-detection logic (keyword/anchor/template/mapping)	Add or extend a <code>run_regression.py</code> fixture covering the <i>pattern</i> , not just the sample document. Re-run the full regression suite — a fix for one supplier must not break others (CLAUDE.md "system fixes, not document fixes")
Confidence/merge/override changes in <code>engine.py</code>	Re-run <code>test_supplier_name_precedence.py</code> and <code>test_supplier_identity_stability.py</code> — these encode the two trickiest invariants
Template matching changes	Re-run <code>test_template_matcher.py</code> , and re-verify the OCR-auto/matching decoupling (§8.3)
DB schema changes	Add a new numbered migration (next is 13); add/extend a <code>database/modules/*</code> JS test against an in-memory DB with that migration applied
UI changes (Settings/Review/Search)	Manual verification in a running <code>npm start</code> session — there is no automated UI test harness. Test the golden path and edge cases (empty states, role-gated buttons for non-admin)
Filing/filename pattern changes	<code>src/modules/filing/test_filename_pattern.js</code> + manual check of <code>.metadata/*.xml</code> output

15. Development Workflow Guidance

This section operationalises the "Working rules" at the top of `CLAUDE.md` for day-to-day development.

15.1 Narrow-scope discipline

- **Read only what the task needs.** The module map in §3 and the "Where to change things" table in §18 are designed so you can jump straight to the relevant file(s) without a repo-wide scan.
- **Stage non-trivial work into incremental edits.** Prefer a focused diff over a broad rewrite — this is easier to review and easier to bisect if a regression appears.

15.2 Extraction/anchoring fixes are system fixes

Per `CLAUDE.md`, **any** change touching field detection, anchors, OCR regions, keyword matching, validation, or supplier/template learning is assumed to be a **reusable application-level** fix until proven otherwise:

- Fix the *layer* (matching strategy, learning rule, normalisation, threshold, validation), not the symptom on the one document you're looking at.
- **No one-document hacks:** no filename-based exceptions, no sample-specific coordinates, no narrow conditionals tuned to a single supplier — unless there's a documented architectural reason (e.g. the `template_field_mappings` system is the sanctioned per-supplier customisation mechanism; ad-hoc code branches are not).
- When you propose a fix, be able to state **explicitly** how it helps *unseen* suppliers/layouts/templates. If it mainly helps the document in front of you, redesign.
- Verify beyond the single failing document — note likely impact on other templates/layouts and regression risk; prefer the regression suite (§14.1) or multiple manual samples over a single-document confirmation.

15.3 Manual verification expectations

- For UI changes, start `npm start` and exercise the feature in the running app — type checking/regression tests verify *code* correctness, not *feature* correctness. State explicitly if you could not test the UI yourself.
- For extraction changes, prefer the fast, deterministic `run_regression.py` fixtures over re-running the full OCR pipeline on a PDF, but do a final OCR-pipeline sanity check on at least one real sample before considering the change done.

15.4 Token-efficient investigation pattern

A pattern that worked well for this manual and is reusable for future large investigations: spawn parallel read-only research agents scoped to non-overlapping file sets (e.g. "Python extraction pipeline", "database layer", "Electron main/IPC/UI"), each asked to cite `file:line` and report facts without editorialising, then synthesise. This keeps the main conversation's context budget for synthesis and writing rather than raw file contents.

16. Build, Packaging & Deployment

16.1 Dev startup

```
cd "C:\GIT Projects\DocuSnap"
npm start          # electron . - uses system Python (py -3.12) + system Tesseract
```

- Python: `py -3.12 <script>.py`
- Tesseract: hardcoded `C:\Program Files\Tesseract-OCR\tesseract.exe` (dev-only path in `ctx.tesseractPath()`)
- DB: `{userData}/docusnap.db` — delete (plus `-wal / -shm`) to reset

16.2 Build

```
npm run build      # electron-builder --win --x64
```

- Output: `dist/DocuSnap Setup <version>.exe` (NSIS installer, Windows x64 only, custom `installer.nsh`)
- `postinstall`: `electron-builder install-app-deps` — rebuilds native addons (`better-sqlite3`, `argon2`) against Electron's ABI
- `extraResources` bundle `python_backend/`, `config/`, and `vendor/` (bundled Python venv + Tesseract, **(inferred)** — confirm exact `vendor/` layout in `package.json`'s `build.extraResources` before changing packaging paths)

- Native `.node` addons are unpacked from the asar archive (required for native bindings to load)
- Code signing is **disabled** (`sign: null`) — installer will trigger Windows SmartScreen warnings; this is a known/accepted state, not a bug

16.3 Packaged differences recap

See [§2](#) "Dev vs. packaged differences" table. The two load-bearing differences to remember when debugging a packaged-only issue:

1. **Python executable path** — `vendor/python/python.exe` vs. `py -3.12` 2. **Tesseract path** — bundled vs. hardcoded `Program Files` path

16.4 Windows path issues

- `render/pages.py`'s `_win_long_path()` (`\\?\\` prefix) is required because Win32 silently strips trailing dots/spaces from path components — relevant for any future code that constructs paths from **supplier names** (which become folder names, [§5](#)) since supplier names commonly end in `Ltd.`, `Inc.`, etc.
- `filing/handler.js` `sanitizeFolderName()` strips Windows-forbidden characters (`\\:”*?<>|`) and caps folder names at 60 chars — any new filing/path-construction code should reuse this rather than re-implementing sanitisation.

17. Known Gotchas / Sharp Edges

1. **Supplier identity drift** ([§6.4](#)): `supplier_name` from Stage 0/logo match is *provisional*. Always use the **re-resolved** `_supplier_name` (after Stages 1–2) for any learning-corpus writes. Writing against the provisional value silently poisons hints/anchors/logos under the wrong supplier key.
2. **Template vs. anchor conflicts** ([§6.3](#)): Three different "taught/curated" sources can compete for the same field — `template_fixed/` `template_anchor` (Stage 0), `template_mapping/` `template_mapping_expanded` (Stage 0.5), and `anchor_crop` (Stage 2). The precedence rules are intentionally asymmetric (mappings *always* beat Stage 0 generics; `anchor_crop` beats *non-taught* results but contends on confidence against other taught results). Changing one merge rule without re-running `test_supplier_name_precedence.py` and `test_supplier_identity_stability.py` risks silently reversing this precedence.
3. **Confidence/override pitfalls**: confidence values are capped at multiple points (95/96/97/45 depending on stage — see [§6.2](#)). Stage 4/4.5 can only **lower** confidence, never raise it. If a field "should" pass review but doesn't, check whether Stage 4.5's format-anomaly cap (45) or Stage 4's maths/date checks fired — `validation_note` will say why.
4. **Windows trailing-dot/space path issue**: any path built from a supplier name (folder names under `OutputRoot/`) can end in `./space` and silently get mangled by Win32 unless the `\\?\\` long-path form is used. Currently handled in `render/pages.py`; if filing/XML-writing code ever needs to *open* (not just create) such a path, apply the same fix.
5. **Renderer errors are invisible in `processing.log`** — see [§13.2](#). Don't assume "nothing in the log" means "nothing happened"; check DevTools.
6. **Stale `__pycache__`** after editing Python files in dev — delete `python_backend/**/__pycache__` if changes don't take effect. Note the tracked `.pyc` file currently showing in `git status` ([§13.4](#)).
7. `getReviewQueue()` **only shows `status='needs_review'`** — inserting as `pending` is a silent dead end ([§5](#)).
8. **Anchor-clearing vs. teaching race**: if `taught_fields` is omitted or mismatched in the `confirm-review` payload, `saveCorrections()` will delete the anchor that was just taught in the same cycle — anchors will appear to "never stick" ([§7.1](#)).
9. **Deferred source-file deletion**: `_pendingSourceMove` is a single-slot, module-level pending operation. If two confirms happen in very quick succession, the first pending move is flushed (`_runPendingSourceMove`) before scheduling the second — there is no queue. This is fine for normal interactive use but worth knowing if you ever batch-confirm programmatically.
10. `field_format_rules` / **Stage 7 Stage 3 does not exist** — don't assume migration 12 covers it; it was repurposed for `templates.ocr_auto_*` ([§7.5](#), [§12.1](#)).
11. `templates/*.json` **on disk are fixtures/exports, not live config** — the app does not read this directory at runtime. Don't edit these files expecting them to change extraction behaviour; live templates are in the `templates/template_fields/` `template_field_mappings` tables.

12. **better-sqlite3 ABI** — any standalone script that needs DB access must run via `ELECTRON_RUN_AS_NODE=1 electron <script>.js`, not plain `node` (§13.5, §14.2).

18. Where to Change Things

I want to change...	Start here
Review window UI/flow (field layout, confirm validation, zone-OCR teaching tool, OCR Preview)	<code>src/windows/review/{index.html,renderer.js}</code> ; IPC in <code>src/modules/review/handler.js</code>
Search (filters, results list, preview, "Edit in Review")	<code>src/windows/search/*</code> ; IPC in <code>src/modules/search/handler.js</code> ; underlying query in <code>database/modules/documents.js</code> <code>search()</code>
Template matching (Stage 0 logo/keyword matching, acceptance thresholds)	<code>python_backend/extraction/template_matcher.py</code> ; tests in <code>python_backend/tests/test_template_matcher.py</code>
Template field mappings (Stage 0.5 admin anchor→target)	<code>python_backend/extraction/template_mapper.py</code> (extraction), <code>database/modules/templates.js</code> (<code>saveMapping / getMappings / region-hint</code>), <code>src/modules/templates/handler.js</code> (IPC), Settings → Templates UI
OCR preprocessing (skew/threshold/noise, OCR Preview, OCR-auto baseline)	<code>src/modules/review/handler.js</code> <code>get-enhanced-preview</code> ; <code>src/modules/processing/handler.js</code> <code>reprocess-document</code> (param resolution); <code>database/modules/templates.js</code> <code>setOcrAutoParams / setOcrAutoEnabled</code> ; Python-side <code>render/preview_enhance.py</code> (inferred filename — verify exact preview-enhance script path)
Filing paths / filename pattern / XML metadata	<code>src/modules/filing/handler.js</code> (<code>commitDocument</code> , <code>buildXml</code> , <code>sanitiseFolderName</code> , <code>parseDate / formatDate</code> , <code>duplicate resolution</code>); pattern tokens via <code>get-filename-pattern-info / preview-filename-pattern</code> ; Settings → File Naming UI
Settings persistence (key-value settings, document types/fields)	<code>src/modules/settings/handler.js</code> ; <code>database/modules/learning.js</code> <code>getSetting / setSetting</code> ; <code>database/modules/document_types.js</code>
Document search internals	<code>database/modules/documents.js</code> <code>search()</code> (SQL/filters); <code>documents.ocr_text</code> populated at insert/reprocess time
PDF rendering / preview (Review, Search, Template Viewer all share this)	<code>python_backend/render/pages.py</code> (+ <code>render_pages.py</code> CLI wrapper); invoked via <code>get-document-pages</code> in <code>src/modules/review/handler.js</code>
Keyword/regex extraction patterns	<code>config/keyword_patterns.json</code> (<code>document_type_keywords</code> , <code>field_patterns</code>); consumed by <code>python_backend/extraction/keyword.py</code>
Anchor/learning behaviour	<code>database/modules/learning.js</code> (<code>saveAnchor</code> , <code>saveCorrections</code> , <code>getHints</code> , <code>getFieldFormats</code>); extraction-side in <code>python_backend/extraction/anchor.py</code>
Auth/users/roles/audit	<code>src/modules/auth/handler.js</code> , <code>database/modules/auth.js</code> ; login UI in <code>src/windows/login/</code>
Folder watch / auto-import	<code>src/modules/watch/handler.js</code> (<code>classifyPoll</code> , polling, integration with <code>processing.handleFileMessage</code>)
Validation rules (date/currency/maths cross-checks)	<code>python_backend/extraction/validator.py</code>
Format-anomaly detection	<code>python_backend/extraction/format_anomaly_checker.py</code> ; <code>database/modules/learning.js</code> <code>getFieldFormats()</code>
Processing mode (fast/smart/ai) & fast-mode suggestion	<code>src/modules/processing/processing_mode_handler.js</code> ; gating logic in <code>python_backend/extraction/engine.py</code> <code>_should_use_llm</code>
PDF splitting	<code>python_backend/pdf_splitter.py</code> (+ CLI wrapper); IPC <code>split-pdf</code> in <code>src/modules/processing/handler.js</code>

19. Troubleshooting Matrix

Symptom	Likely subsystem	First files / logs to check
App won't start past login	Auth / DB migrations	<code>processing.log</code> (early init errors), terminal stdout from <code>npm start</code> , <code>database/index.js</code> migration order
Document processed but never shows in Review	Insert status bug	<code>_handleFileMessage</code> in <code>src/modules/processing/handler.js</code> , <code>documents.getReviewQueue()</code> filter
'str' object has no attribute 'get' crash	<code>_</code> -key handling	<code>python_backend/process_docs.py</code> (<code>sanitise_extractions</code> , pop order), <code>python_backend/extraction/validator.py</code>
Wrong supplier learned / templates polluted across suppliers	Supplier identity resolution	<code>python_backend/extraction/engine.py</code> re-resolution block, <code>test_supplier_identity_stability.py</code>
Field extraction regresses for one supplier after a "fix"	Merge/override precedence	<code>python_backend/extraction/engine.py</code> Stage 1/2/0.5 merge logic, <code>test_supplier_name_precedence.py</code> , run full <code>run_regression.py</code>
Taught anchor doesn't persist / gets wiped	<code>taught_fields</code> plumbing	<code>src/windows/review/renderer.js</code> (<code>anchorTaughtFields</code>), <code>confirm-review</code> payload, <code>database/modules/learning.js</code> <code>saveCorrections</code>
Reprocess produces worse results than original	OCR-auto params / merge strategy	<code>src/modules/processing/handler.js</code> <code>reprocess-document</code> (param resolution + merge), <code>templates.ocr_auto_params</code>
Template match picks wrong template, or OCR-auto seems to bias matching	Stage 0 matching	<code>python_backend/extraction/template_matcher.py</code> , <code>test_template_matcher.py</code> , confirm <code>templates.getAll()</code> ordering unaffected by <code>ocr_auto_*</code>
File can't be deleted/moved after confirm (EBUSY/EPERM)	Deferred source-file removal	<code>src/modules/review/handler.js</code> (<code>_pendingSourceMove</code> / <code>_scheduleSourceMove</code>), <code>src/modules/filing/handler.js</code> <code>removeSourceFile</code>
PDF preview blank/error for a specific supplier folder	Win32 path edge case	<code>python_backend/render/pages.py</code> <code>_win_long_path</code> , <code>test_pages_long_path.py</code> , check supplier name for trailing <code>.</code> /space
Search returns nothing for text that's visibly on the page	Full-text column not populated	<code>documents.ocr_text</code> (migration 10) — confirm it's written at insert/reprocess time
Field flagged needs_review with a confusing validation_note	Stage 4 / 4.5	<code>python_backend/extraction/validator.py</code> and <code>format_anomaly_checker.py</code> — <code>validation_note</code> text indicates which check fired
Settings change doesn't propagate to other open windows	Broadcast missing	<code>notifyAllWindows</code> calls in the relevant handler (e.g. <code>theme-changed</code> , <code>processing-mode-changed</code>)
Watch folder not picking up new files	Polling/state machine	<code>src/modules/watch/handler.js</code> <code>classifyPoll</code> , check file extension against <code>SUPPORTED_EXTENSIONS</code> , 30s stability delay
Login locked out / "try again later"	Rate limiting	<code>src/modules/auth/handler.js</code> <code>_checkRateLimit</code> / <code>_recordFailedAttempt</code> — per-username delay ladder, wait or use recovery code
better-sqlite3 native module error	ABI mismatch	Ensure <code>npm run postinstall</code> (electron-builder <code>install-app-deps</code>) has run; any standalone script must run via <code>ELECTRON_RUN_AS_NODE=1 electron</code>

20. Extension Roadmap Guidance

These are directions referenced in CLAUDE.md or implied by the current architecture's seams. None of these should require a broad refactor — each has a natural extension point already identified above.

20.1 Detached search client

See §11.4. Extract `search-documents` / `get-document-with-extractions` / `get-document-pages` behind a small read-only API contract; keep `requireRole` checks in the main process and design a separate auth boundary for the detached client rather than weakening existing checks.

20.2 Localization

No i18n framework currently exists — all UI strings are inline in `*/index.html` / `*/renderer.js`. A safe incremental approach: introduce a simple key→string lookup (per window or shared in `src/windows/shared/`) and migrate one window at a time, starting with the smallest (Login, 7 screens) as a proof of concept before tackling Review/Settings/Search.

20.3 Workflow / approval features

The Search window already has a **placeholder "Workflow" section** (§10.5, `search-actions.js`). A natural extension: add a `documents.workflow_status` column (new migration) and corresponding IPC handlers in `search/handler.js` or a new `workflow/handler.js` module, following the existing role-gated-handler pattern from `auth/handler.js`'s `requireRole`. Audit every state transition via `logAudit()` (§7.6) the same way `search_open_review` is logged.

20.4 Audit enhancements

`audit_log` already captures auth + search-navigation events. Extending coverage to confirm/defer/delete/reprocess actions would follow the same `logAudit(db, {action, target_type: 'document', target_id, details})` pattern — add calls at the relevant points in `review/handler.js` and `processing/handler.js` without changing the `audit_log` schema.

20.5 OCR rule refinements

- **Per-field OCR-auto params** (currently template-wide): would need a new column/table scoped to `(template_id, field_key)` rather than changing `templates.ocr_auto_params`'s shape — keep the existing template-wide baseline as a fallback for fields without an override.
- **Stage 7 Stage 3** (`field_format_rules`, §7.5): next migration is **13**; write path would extend `learning.saveCorrections()`'s transaction; read path needs a new `--format-rules-file` CLI arg threaded through `buildTrainingArgs()` in `processing/handler.js` and consumed in `engine.py` to override the in-memory `format_class_index` once `confirmed_count >= 10`.

In all cases, follow §15.2: new learning/extraction behaviour must be scoped strictly by `(supplier_name, document_type, field_key)` (or documented global fallback like `__global__`), with conservative confirmed-sample minimums, mirroring every existing learning corpus in §7.

End of manual.